

Lecture 31. Computing the SVD

The computation of the SVD of an arbitrary matrix can be reduced to the computation of the eigenvalue decomposition of a hermitian square matrix, but the most obvious way of doing this is not stable. Instead, the standard methods for computing the SVD are based implicitly on another kind of reduction to hermitian form. For speed, the matrix is first unitarily bidiagonalized.

SVD of A and Eigenvalues of A^*A

As stated in Theorem 5.4, the SVD of the $m \times n$ matrix A ($m \geq n$), $A = U\Sigma V^*$, is related to the eigenvalue decomposition of the matrix A^*A ,

$$A^*A = V\Sigma^*\Sigma V^*.$$

Thus, mathematically speaking, we might calculate the SVD of A as follows:

1. Form A^*A ;
2. Compute the eigenvalue decomposition $A^*A = V\Lambda V^*$;
3. Let Σ be the $m \times n$ nonnegative diagonal square root of Λ ;
4. Solve the system $U\Sigma = AV$ for unitary U (e.g., via QR factorization).

This algorithm is frequently used, often by people who have rediscovered the SVD for themselves. The matrix A^*A is known as the *covariance matrix* of A , and it has familiar interpretations in statistics and other fields. The algorithm

is unstable, however, because it reduces the SVD problem to an eigenvalue problem that may be much more sensitive to perturbations.

The difficulty can be explained as follows. We have seen that when a hermitian matrix A^*A is perturbed by δB , the absolute changes in each eigenvalue are bounded by the 2-norm of the perturbation. By Exercise 26.3(b), $|\lambda_k(A^*A + \delta B) - \lambda_k(A^*A)| \leq \|\delta B\|_2$. As is implied by equation (31.2) below, a similar bound holds for the singular values of A itself, $|\sigma_k(A + \delta A) - \sigma_k(A)| \leq \|\delta A\|_2$. Thus a backward stable algorithm for computing singular values would obtain $\tilde{\sigma}_k$ satisfying

$$\tilde{\sigma}_k = \sigma_k(A + \delta A), \quad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}), \quad (31.1)$$

which would imply

$$|\tilde{\sigma}_k - \sigma_k| = O(\epsilon_{\text{machine}} \|A\|).$$

Now observe what happens if we proceed by computing $\lambda_k(A^*A)$. If $\lambda_k(A^*A)$ is computed stably, we must expect errors of the order

$$|\tilde{\lambda}_k - \lambda_k| = O(\epsilon_{\text{machine}} \|A^*A\|) = O(\epsilon_{\text{machine}} \|A\|^2).$$

Square-rooting to get σ_k , we find

$$|\tilde{\sigma}_k - \sigma_k| = O(|\tilde{\lambda}_k - \lambda_k| / \sqrt{\lambda_k}) = O(\epsilon_{\text{machine}} \|A\|^2 / \sigma_k).$$

This is worse than the previous result by a factor $O(\|A\|/\sigma_k)$. This is no problem for the dominant singular values of A , with $\sigma_k \approx \|A\|$, but it is a big problem for any singular values with $\sigma_k \ll \|A\|$. For the smallest singular value σ_n , we must expect a loss of accuracy of order $\kappa(A)$ —a “squaring of the condition number,” just as in the use of the normal equations for certain least squares problems (Lecture 19).

A Different Reduction to an Eigenvalue Problem

There is an alternative, stable way to reduce the SVD to an eigenvalue problem. Assume that A is square, with $m = n$; this is no essential restriction, since we shall see that rectangular singular value problems can be reduced to square ones. Consider the $2m \times 2m$ hermitian matrix

$$H = \begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix} \quad (31.2)$$

mentioned earlier in Exercise 5.4. Since $A = U\Sigma V^*$ implies $AV = U\Sigma$ and $A^*U = V\Sigma^* = V\Sigma$, we have the block 2×2 equation

$$\begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix} \begin{bmatrix} V & V \\ U & -U \end{bmatrix} = \begin{bmatrix} V & V \\ U & -U \end{bmatrix} \begin{bmatrix} \Sigma & 0 \\ 0 & -\Sigma \end{bmatrix}, \quad (31.3)$$

which amounts to an eigenvalue decomposition of H . Thus we see that the singular values of A are the absolute values of the eigenvalues of H , and the singular vectors of A can be extracted from the eigenvectors of H .

Thus one could obtain the SVD of a square matrix A by forming the matrix H and computing its eigenvalue decomposition. In contrast to the use of AA^* or A^*A , this approach is stable. The standard algorithms for the SVD are based on this idea, though in a disguised manner in which no matrices of dimension as large as $m + n$ are formed explicitly. And the key step to make the process fast is an initial unitary reduction to bidiagonal form.

Two Phases

We have seen that hermitian eigenvalue problems are usually solved by a two-phase computation: first reduce the matrix to tridiagonal form, then diagonalize the tridiagonal matrix. Since the work of Golub, Kahan, and others in the 1960s, an analogous two-phase approach has been standard for the SVD. The matrix is brought into bidiagonal form, and then the bidiagonal matrix is diagonalized:

$$\begin{array}{ccc} \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix} & \xrightarrow{\text{Phase 1}} & \begin{bmatrix} \times & \times & & \\ & \times & \times & \\ & & \times & \times \\ & & & \times \end{bmatrix} \\ A & & B \\ & & \xrightarrow{\text{Phase 2}} & \begin{bmatrix} \times & & & \\ & \times & & \\ & & \times & \\ & & & \times \end{bmatrix} \\ & & \Sigma \end{array}$$

Phase 1 involves a finite number of operations, $O(mn^2)$ flops. Phase 2 in principle requires an infinite number of operations, but the standard algorithms converge superlinearly, and thus only $O(n \log(|\log(\epsilon_{\text{machine}})|))$ iterations are needed for convergence to order $\epsilon_{\text{machine}}$ (Exercise 25.2). In practice, we think of $\epsilon_{\text{machine}}$ as a constant and say that convergence is achieved in $O(n)$ iterations. Because the matrix operated on is bidiagonal, each of these iterations requires only $O(n)$ flops. Phase 2 therefore requires $O(n^2)$ flops all together (assuming singular values but not vectors are required). Thus, although Phase 1 is finite and Phase 2 is in principle infinite, in practice the latter is much the less expensive, just as we found for the symmetric eigenvalue problem.

Golub–Kahan Bidiagonalization

In Phase 1 of the SVD computation, we bring A into bidiagonal form by applying distinct unitary operations on the left and right. Note how this differs from the computation of eigenvalues, where the same unitary operations must be applied on both sides so that each step is a similarity transformation. In

that case, it was only possible to introduce zeros below the first subdiagonal. Here, we are able to completely triangularize and also introduce zeros above the first superdiagonal.

The simplest method for accomplishing this, *Golub–Kahan bidiagonalization*, proceeds as follows. Householder reflectors are applied alternately on the left and the right. Each left reflection introduces a column of zeros below the diagonal. The right reflection that follows introduces a row of zeros to the right of the first superdiagonal, leaving intact the zeros just introduced in the column. For example, for a 6×4 matrix, the first two pairs of reflections look like this:

$$\begin{array}{ccc}
 \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix} & \xrightarrow{U_1^*} & \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix} & \xrightarrow{\cdot V_1} & \begin{bmatrix} \times & \times & 0 & 0 \\ & \times & \times & \times \\ & \times & \times & \times \\ & \times & \times & \times \\ & \times & \times & \times \\ & \times & \times & \times \end{bmatrix} \\
 A & & U_1^* A & & U_1^* A V_1 \\
 & & & & \\
 & & \xrightarrow{U_2^*} & & \xrightarrow{\cdot V_2} \\
 & & \begin{bmatrix} \times & \times & & \\ & \times & \times & \times \\ & 0 & \times & \times \\ & 0 & \times & \times \\ & 0 & \times & \times \\ & 0 & \times & \times \end{bmatrix} & & \begin{bmatrix} \times & \times & & \\ & \times & \times & 0 \\ & & \times & \times \\ & & \times & \times \\ & & \times & \times \\ & & \times & \times \end{bmatrix} \\
 & & U_2^* U_1^* A V_1 & & U_2^* U_1^* A V_1 V_2
 \end{array}$$

The left-multiplication by U_1^* modifies rows 1 to m , introducing zeros in column 1 below the diagonal. The right-multiplication by V_1 modifies columns 2 to n , introducing zeros in row 1 without destroying the zeros in column 1. The process continues with operations on rows 2 to m , then columns 3 to n , and so on.

At the end of this process, n reflectors have been applied on the left and $n - 2$ on the right. The pattern of floating point operations resembles two Householder QR factorizations interleaved with each other, one operating on the $m \times n$ matrix A , the other on the $n \times m$ matrix A^* . The total operation count is therefore twice that of a QR factorization (10.9), i.e.,

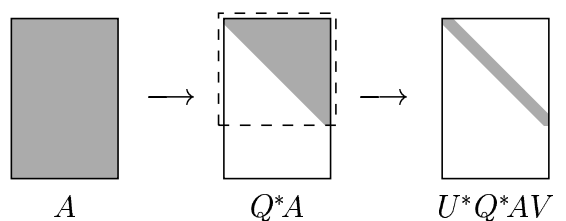
$$\text{Work for Golub–Kahan bidiagonalization: } \sim 4mn^2 - \frac{4}{3}n^3 \text{ flops.} \quad (31.4)$$

Faster Methods for Phase 1

For $m \gg n$, this operation count is unnecessarily large. A single QR factorization would introduce zeros everywhere below the diagonal, and for $m \gg n$,

these are the great majority of the zeros that are needed. Yet the operation count for the Golub–Kahan method is twice as high. This observation suggests an alternative method for bidiagonalization with $m \gg n$, first proposed by Lawson and Hanson and later developed by Chan. The idea, *LHC bidiagonalization*, is illustrated as follows:

Lawson–Hanson–Chan bidiagonalization



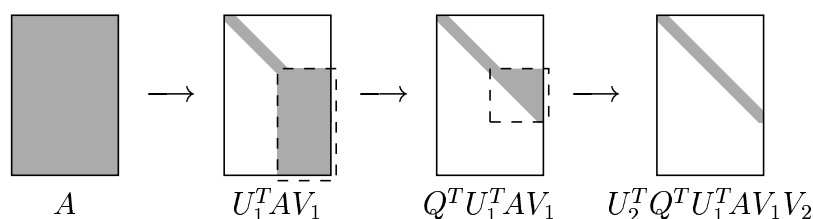
We begin by computing the QR factorization $A = QR$. Then we compute the Golub–Kahan bidiagonalization $B = U^*RV$ of R . The QR factorization requires $2mn^2 - \frac{2}{3}n^3$ flops (10.9), and the Golub–Kahan procedure, which now only has to operate on the upper $n \times n$ submatrix, requires $\frac{8}{3}n^3$ flops. The total operation count is

$$\text{Work for LHC bidiagonalization: } \sim 2mn^2 + 2n^3 \text{ flops.} \quad (31.5)$$

This is cheaper than Golub–Kahan bidiagonalization for $m > \frac{5}{3}n$ (Exercise 31.1). Curiously, the LHC procedure creates zeros and then destroys them again (in the lower triangle of the upper $n \times n$ square of A), but there is a net gain.

The LHC procedure is advantageous only when $m > \frac{5}{3}n$, but the idea can be generalized so as to realize a saving for any $m > n$. The trick is to apply the QR factorization not at the beginning of the computation, but at a suitable point in the middle. This is advantageous because in the Golub–Kahan process, a matrix with $m > n$ becomes skinnier as the bidiagonalization proceeds. If the initial aspect ratio is, say, $m/n = 3/2$, it will steadily grow to $5/3$ and 2 and beyond. After step k , the aspect ratio of the remaining matrix is $(m - k)/(n - k)$, and when this figure gets sufficiently large, it makes sense to perform a QR factorization to reduce the problem to a square matrix.

Three-step bidiagonalization



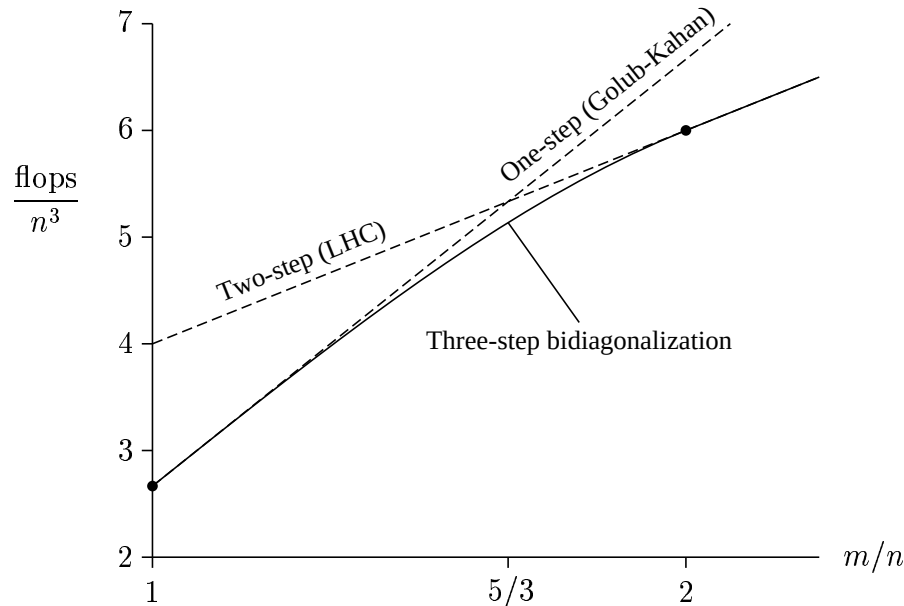


Figure 31.1. Operation counts for three bidiagonalization algorithms applied to $m \times n$ matrices, from (31.4), (31.5), and (31.6). Three-step bidiagonalization provides a pleasingly smooth interpolant between the other two methods, though the improvement is hardly large.

When should the QR factorization be performed? If we aim solely to minimize the operation count, the answer is simple: when the aspect ratio reaches $(m - k)/(n - k) = 2$ (Exercise 31.2). This choice leads to the formula

$$\text{Work for three-step bidiagonalization: } \sim 4mn^2 - \frac{4}{3}n^3 - \frac{2}{3}(m - n)^3 \text{ flops,} \quad (31.6)$$

a modest improvement over the other two methods for $n < m < 2n$.

The operation counts for the three methods are plotted as a function of m/n in Figure 31.1. It must be admitted that the improvement achieved by the three-step method is small enough that in practice, other matters besides the count may determine which method is best on a real machine (see p. 59).

Phase 2

In Phase 2 of the computation of the SVD, the SVD of the bidiagonal matrix B is determined. From the 1960s to the 1990s, the standard algorithm for this was a variant of the QR algorithm. More recently, divide-and-conquer algorithms have also become competitive, and in the future, they are likely to become the standard. We shall not give details.

Exercises

31.1. (a) Show that, as claimed in the text and illustrated in Figure 31.1, the crossover aspect ratio at which LHC bidiagonalization begins to beat Golub–Kahan bidiagonalization is $m/n = 5/3$.

(b) By what fraction is three-step bidiagonalization faster than the other two methods for $m/n = 5/3$?

31.2. Show that in three-step bidiagonalization, the optimal point at which to perform the QR factorization is when the matrix reaches an aspect ratio of 2.

31.3. Show that if the entries on both principal diagonals of a bidiagonal matrix are all nonzero, then the singular values of the matrix are distinct. (See Exercise 25.1.)

31.4. Let A be the $m \times m$ upper-triangular matrix with 0.1 on the main diagonal and 1 everywhere above the diagonal. Write a program to compute the smallest singular value of A in two ways: by calling a standard SVD software, and by forming A^*A and computing the square root of its smallest eigenvalue. Run your program for $1 \leq m \leq 30$ and plot the results as two curves on a log scale. Do the results conform to our general discussion of these algorithms?

31.5. Let A be an $m \times n$ matrix whose entries are independent samples from $N(0, 1)$, the normal distribution of mean 0, variance 1 (compare Exercise 12.3). Let B be a bidiagonal matrix

$$B = \begin{bmatrix} x_m & y_{n-1} & & & \\ & x_{m-1} & y_{n-2} & & \\ & & \ddots & \ddots & \\ & & & x_{m-(n-2)} & y_1 \\ & & & & x_{m-(n-1)} \end{bmatrix},$$

where each x or y is the positive square root of an independent sample from the χ^2 distribution with degree equal to the attached subscript. (The χ^2 distribution of degree k is equal to the distribution of the sum of squares of k independent variables from $N(0, 1)$.)

(a) Show that the distributions of the singular values of A and B are the same.

(b) Verify this result by an experiment. Specifically, take $m = 100$ and $n = 50$, construct random matrices A and B as indicated, and plot the singular values of A against those of B .